TP1 : environnement et premières complétions

Durée : 25 min **Partie** : P1 (découverte et installation)

Objectif

Configurer les outils, puis faire écrire trois fonctions à Copilot. Savoir préciser une demande et vérifier la proposition obtenue.

Prérequis

- Démo 1 ; VS Code récent, Python 3.12 ou plus, uv, comptes GitHub et ChatGPT.
- Conserver la copie complète du dépôt ; ouvrir `tps/tp1_environnement/` seul dans VS Code.
- Utiliser uniquement les fichiers fictifs fournis. Ne pas modifier `test_premiers_pas.py`.

Partie 1 — Préparer les outils (10 min)

1. Activer Copilot depuis la barre d'état (**Use AI Features**) et se connecter à GitHub. Vérifier l'allocation disponible dans son tableau de bord.
2. Depuis **Preferences: Open User Settings (JSON)**, lire `settings_recommandes.jsonc` et reporter les réglages utiles.
3. Dans ChatGPT, repérer **Settings > Data controls > Improve the model for everyone** et le chat temporaire, selon les règles de votre entreprise.
4. Lancer `uv sync` à la racine du dépôt, puis, dans le dossier du TP, `uv run pytest -q`. Les 18 tests doivent être collectés ; les fonctions sont encore à compléter.

Partie 2 — Guider les complétions (10 min)

1. Ouvrir `premiers_pas.py`. Pour `est_palindrome`, `formater_euros` et `est_email_valide`, supprimer `raise NotImplementedError`, laisser Copilot proposer le corps et relire avant d'accepter avec `Tab`.
2. Lancer `uv run pytest -q` après chaque fonction. En cas d'échec, préciser la docstring avec le résultat attendu, puis demander une nouvelle proposition.
3. Conserver une consigne que vous avez précisée et expliquer son effet.

Exemple de contexte à donner à Copilot si le format des euros pose problème :

```
# Exemples du contrat :
# formater_euros(1234.5) == "1 234,50 EUR"
# formater_euros(999.999) == "1 000,00 EUR"
# Indice Python : f"{1234.5:,.2f}" produit "1,234.50".
# Adapter les séparateurs au format demandé après l'arrondi.
```

Ce bloc aide à formuler la demande. Faire proposer l'implémentation par Copilot, puis la vérifier avec les tests.

Partie 3 — Comparer avec ChatGPT (3 min)

Coller `formater_euros` avec sa docstring et demander :

> Explique cette fonction et propose trois cas de test supplémentaires. Justifie chaque résultat attendu avec la docstring.

Vérifier un cas proposé à la main. Noter une différence avec Copilot dans l'éditeur.

Partie 4 — Vérifier et expliquer (2 min)

Lancer `uv run pytest -q`. Montrer une consigne utilisée et le test qui vérifie le résultat.

Critères de réussite

- Les 18 tests passent, sans modification du fichier de tests.
- Les trois fonctions ont été proposées par Copilot et relues.
- Vous pouvez expliquer comment une signature, une docstring ou un exemple a guidé la proposition.

Corrigé

`corrige/premiers_pas.py` et `corrige/prompts_types.md`. Une autre implémentation qui respecte le contrat et passe les tests est valable.

TP2 : un module de réservation de salles avec Copilot

Durée : 65 min **Partie** : P2 (complétion, tests, débogage et refactoring)

Objectif

Compléter un module, faire générer des tests depuis ses exigences, corriger un bug, refactoriser et importer des réservations CSV. Relire chaque proposition avant de la garder.

Prérequis

- Démo 2 et TP1 ; environnement commun préparé avec `uv sync`.
- Ouvrir `tps/tp2_module_reservations/` seul dans VS Code.
- Ne modifier ni `test_acceptation_tp2.py` ni `test_import_csv.py`.

Partie 1 — Écrire avec les complétions (15 min)

Dans `reservations.py`, faire compléter `chevauche`, `ajouter` et `taux_occupation` à partir des docstrings. Garder les tests ouverts comme contexte. Si une proposition est fausse, préciser la règle ou utiliser l'inline chat.

```
uv run pytest -q test_acceptation_tp2.py -k "chevauche or ajouter or taux"
```

Pour représenter deux créneaux qui se touchent, utiliser ce contexte dans votre demande :

```
jour = date(2026, 10, 12)
a = Reservation("Turing", datetime(2026, 10, 12, 9),
                datetime(2026, 10, 12, 10), "Alice")
b = Reservation("Turing", datetime(2026, 10, 12, 10),
                datetime(2026, 10, 12, 11), "Bruno")
# Selon le contrat, chevauche(a, b) doit renvoyer False.
```

Partie 2 — Générer des tests depuis les exigences (10 min)

Demander à Copilot des tests dans `test_reservations.py`, à partir des docstrings. Exiger au moins huit cas, dont un planning vide, deux créneaux qui se touchent et un horaire invalide. Relire les valeurs attendues avant de lancer les tests : elles doivent venir des règles métier.

Exemple de test à suivre :

```
def test_creneau_qui_depasse_la_fermeture():
    reservation = Reservation("Turing", datetime(2026, 10, 12, 17),
                              datetime(2026, 10, 12, 19), "Alice")
    with pytest.raises(ValueError):
        ajouter([], reservation)
```

Partie 3 — Corriger le bug des créneaux libres (10 min)

Lancer `uv run pytest -q test_acceptation_tp2.py`. Donner à Copilot le test `test_creneaux_libres_fin_de_journee`, la docstring et le symptôme. Faire corriger l'implémentation, puis expliquer en une phrase pourquoi le dernier créneau manquait. Ne pas modifier le test.

Partie 4 — Refactoriser sous filet de tests (15 min)

Vérifier que `test_rapport_hebdomadaire_inchange` passe. Faire proposer un plan pour simplifier `rapport_hebdomadaire`, puis faire appliquer ce plan : noms explicites, constantes, calcul commun avec `taux_occupation`. Relire le diff et lancer `uv run pytest -q test_acceptation_tp2.py test_reservations.py`. Le résultat du rapport doit rester identique.

Partie 5 — Écrire les règles du projet (5 min)

Faire créer `AGENTS.md` avec les sections Projet, Commandes, Conventions et Tests. Vérifier chaque commande. Ajouter une règle propre au projet avec sa raison, un exemple de test à suivre et l'interdiction de modifier les tests fournis.

Partie 6 — Importer les réservations CSV (10 min)

Faire compléter `charger_csv` en réutilisant `ajouter` pour valider chaque réservation. Le bloc suivant peut servir de point de départ : le faire relire et expliquer par Copilot.

```
import csv

def charger_csv(chemin):
    planning = []
    with open(chemin, encoding="utf-8", newline="") as fichier:
        for ligne in csv.DictReader(fichier, delimiter=";"):
            reservation = Reservation(
                ligne["salle"], datetime.fromisoformat(ligne["debut"]),
                datetime.fromisoformat(ligne["fin"]), ligne["formateur"])
            planning = ajouter(planning, reservation)
    return planning
```

Lancer `uv run pytest -q`. Vérifier que le CSV normal est trié et que `reservations_conflit.csv` provoque bien `ConflitDeReservation`.

Critères de réussite

- Les 19 tests fournis passent : 17 sur le module, 2 sur l'import CSV.
- Vos huit tests ou plus passent ; leurs valeurs attendues viennent des exigences.
- Le rapport reste inchangé après refactoring, et `AGENTS.md` contient des commandes vérifiées.
- Vous pouvez expliquer une correction et sa vérification.

Corrigé

`corrige/reservations.py`, `corrige/test_reservations.py`, `corrige/AGENTS.md` et `corrige/prompts_types.md`.

TP3 : développer avec Codex et intégrer l'API dans une boutique

Durée : 50 min **Partie** : P3 (revue, débogage, tests, documentation et API)

Objectif

Piloter Codex dans VS Code : lui faire lire un projet, reproduire un bug, modifier les fichiers et lancer les tests. Relire ses changements, puis lui faire intégrer l'API OpenAI pour préparer la fiche d'un nouvel article de boutique.

Prérequis

- Démos 3 et 4 ; extension Codex installée et connectée au compte ChatGPT ; environnement commun préparé avec `uv sync`.
- Ouvrir `tps/tp3_codex_api/` seul dans VS Code.
- Ouvrir Codex par son icône ou par **Codex: Open Codex Sidebar** dans la palette de commandes. Travailler localement. Vérifier le contrôle des permissions sous la saisie : **Ask for approval** si disponible. Ce mode autorise les actions dans le dossier et demande avant de dépasser ce périmètre ; chaque commande locale ne déclenche pas nécessairement une demande.
- Lire `AGENTS.md`. Utiliser les données fictives fournies, conserver une copie de départ et ne pas modifier les tests d'acceptation ni consulter `corrige/` pendant le travail.
- Pour l'appel réel, clé de formation dans `OPENAI_API_KEY` (voir le README de la formation). Les tests et le client simulé permettent de poursuivre en cas de problème d'accès.

Partie 1 — Faire une revue ciblée (7 min)

Ouvrir `legacy/notes_frais.py` et `SPEC.md`. Avant de solliciter Codex, repérer le mot de passe fictif et remplacer sa valeur par `***`. Demander ensuite dans le panneau Codex :

> Lis AGENTS.md, SPEC.md et legacy/notes_frais.py. Sans modifier de fichier, relève les écarts à la spécification dans un tableau : ligne, règle concernée, gravité, correction proposée. Propose un plan court et indique les commandes pour reproduire puis vérifier les problèmes. Ne consulte pas corrige/.

Vérifier deux remarques dans les fichiers et ajuster le plan. Codex a accès au projet : vérifier les fichiers qu'il a lus et les actions qu'il annonce.

Partie 2 — Reproduire et corriger les écarts (12 min)

> Exécute uv run python legacy/notes_frais.py donnees/notes_septembre.csv et uv run pytest -q test_acceptation_tp3.py. Explique les échecs observés par rapport à SPEC.md, puis applique les corrections du plan dans legacy/notes_frais.py uniquement. Retire le mot de passe inutilisé, remplace eval, puis relance ces commandes. Ne modifie pas les tests fournis.

Lire les commandes et leurs sorties dans la conversation. Si une autorisation est demandée, vérifier sa cible avant de l'accorder. Pour remplacer `eval`, ce modèle peut aider Codex :

```
import json
from pathlib import Path

fichier_taux = Path(__file__).resolve().parent.parent / "taux.json"
with fichier_taux.open(encoding="utf-8") as fichier:
    taux = json.load(fichier)
```

Faire corriger la conversion avant plafond, la virgule décimale, le rejet signalé d'une catégorie inconnue et l'affichage français. Pour traiter un montant, utiliser ce point de départ :

```
montant = float(texte.strip().replace(",", "."))
en_euros = montant * taux[devise]
# Appliquer ensuite le plafond de la catégorie, puis arrondir au centime.
```

Ouvrir les différences proposées par Codex et expliquer une correction avant de la conserver. Relancer soi-même `uv run pytest -q test_acceptation_tp3.py` dans le terminal. Les totaux attendus sont Diallo 48,92 EUR, Martin 180,30 EUR et Nguyen 251,40 EUR ; la dépense « taxi » est rejetée avec un motif.

Partie 3 — Écrire les tests depuis la spécification (8 min)

> À partir des règles et exemples de SPEC.md, propose d'abord un tableau de cas et de résultats attendus. Après ma relecture, crée test_notes_frais.py avec ces tests pytest, adapte les appels aux fonctions corrigées et exécute les tests. Ne déduis pas les valeurs attendues du code et ne change pas les tests fournis.

Vérifier à la main trois cas avant la création du fichier : conversion avant plafond, montant à virgule et catégorie inconnue.

```
# Exemple d'attendu tiré des règles, à adapter au nom de votre fonction :
assert montant_rembourse("hotel", 110, "GBP", {"GBP": 1.17}) == 120.0
```

Lancer `uv run pytest -q test_acceptation_tp3.py test_notes_frais.py`.

Partie 4 — Documenter le résultat (3 min)

Demander à Codex d'ajouter les docstrings et de créer `README.md` avec une section « Utilisation » de dix lignes maximum. Lui demander de vérifier la commande et la sortie en exécutant le script, sans inventer d'option. Relire les fichiers modifiés.

Partie 5 — Générer la fiche d'un nouvel article (15 min)

La boutique possède `catalogue.json`. À l'ajout de `nouvel_article.json`, elle prépare sa fiche en brouillon. Les fiches existantes doivent rester intactes ; une référence déjà présente est refusée avant l'appel API. La démo 4 a montré le même mécanisme d'intégration sur le tri d'un ticket support.

1. Lire `ArticlePublic`, `FicheProduit` et les deux fonctions à compléter dans `catalogue.py`.
2. Demander à Codex de compléter uniquement `generer_fiche` et `ajouter_article`, puis de lancer `uv run pytest -q test_api_catalogue.py` avec le client simulé, sans appel réel. Le client est reçu en paramètre. Le bloc suivant montre la structure de l'appel ; expliquer chaque argument avant de le garder :

```
reponse = client.responses.parse(
    model=modele,
    instructions=INSTRUCTIONS,
    input=json.dumps(donnees, ensure_ascii=False),
    text_format=FicheProduit,
    store=False,
)
if reponse.output_parsed is None:
    raise ValueError("Aucune fiche structurée : refus ou réponse incomplète.")
return FicheProduit.model_validate(reponse.output_parsed)
```

3. Relire les changements de Codex, notamment la fin d'`ajouter_article`, après le contrôle des doublons. Exemple de construction :

```
fiche = generer_fiche(public, client, modele)
nouveau = {**public, "fiche": fiche.model_dump(), "statut": "brouillon"}
return [*catalogue, nouveau]
```

4. Relancer soi-même `uv run pytest -q test_api_catalogue.py` : les huit tests utilisent un client simulé. Avec la clé, lancer dans son terminal `uv run python catalogue.py catalogue.json nouvel_article.json`, sans transmettre la clé dans la conversation Codex. Lire `catalogue_brouillon.json` et comparer chaque affirmation aux caractéristiques. En cas d'accès API indisponible, vérifier le même parcours avec les tests et le formateur. Aucun site n'est publié par cet exercice.

Partie 6 — Vérifier et expliquer (5 min)

Lancer soi-même `uv run pytest -q`. Montrer une correction de Codex avec sa règle et son test, puis un changement que l'on a vérifié ou fait reprendre. Expliquer ce qui déclenche l'appel API, ce qui est envoyé et pourquoi le texte généré reste à relire.

Critères de réussite

- Les 15 tests fournis passent : 7 pour les notes de frais et 8 pour le catalogue ; vos tests passent aussi.
- Le README décrit une commande et une sortie vérifiées.
- Le nouvel article reçoit une fiche en brouillon ; les anciens articles sont conservés ; un doublon ne provoque aucun appel.
- Vous avez vérifié les commandes exécutées et les différences produites par Codex.
- Vous distinguez Codex, qui développe le projet, et l'API, que l'application appelle pour générer une fiche.

Corrigé

`corrige/legacy/notes_frais.py`, `corrige/catalogue.py`, `corrige/test_notes_frais.py`, `corrige/README.md` et `corrige/prompts_types.md`.

TP4 : générateur de changelog et charte d'usage

Durée : 60 min **Partie** : P4 (combiner les outils et vérifier le résultat)

Objectif

Construire un générateur de CHANGELOG testé, créer son fichier de prompt réutilisable et rédiger une charte d'usage. Répartir les tâches entre ChatGPT et Copilot, puis justifier les décisions prises.

Prérequis

- Démo 5 et TP2 ; environnement préparé avec `uv sync`.
- Ouvrir `tps/tp4_projet_complet/` seul dans VS Code. Choisir la cible **Local** pour les fichiers `.prompt.md`.
- Fichiers fournis : `besoin.md`, `commits.txt`, `changelog.py`, `test_acceptation_tp4.py`, `charte_ia_modele.md`.
- Conserver les signatures de `changelog.py` et les tests d'acceptation. Le jeu `commits.txt` suffit, sans historique Git à créer.

Partie 1 — Cadrer avec ChatGPT (7 min)

Donner `besoin.md` et les docstrings de `changelog.py`. Demander une spécification d'une page maximum : règles, critères vérifiables, cas limites et hors périmètre. Répondre aux questions, relire, puis enregistrer `spec.md`. Vérifier une règle contre une docstring et un test.

Partie 2 — Écrire les règles du projet (5 min)

Faire créer un `AGENTS.md` court. Utiliser ces lignes comme point de départ, puis faire préciser les conventions propres au projet :

```
## Projet
Générateur de changelog Python. Référence : spec.md.
## Commandes
uv run pytest -q
uv run python changelog.py commits.txt 2.0.0 --date 2026-10-09
## Tests
Ne pas modifier test_acceptation_tp4.py ni les signatures fournies.
Les résultats attendus viennent des règles, pas du code existant.
```

Vérifier chaque commande avant de garder le fichier.

Partie 3 — Planifier, implémenter et tester (23 min)

Demander un plan à Copilot pour implémenter `spec.md` dans `changelog.py`, avec des tests dans `test_changelog.py`. Relire le plan, puis faire appliquer les étapes. Relire les diffs et lancer les tests après les changements.

Si l'analyse d'une ligne est délicate, donner cet exemple de découpage à l'assistant :

```
ligne = "9f2c1ab|2026-10-08|feat(api)!: supprime la route /v1"
champs = ligne.strip().split("|", 2)
# Vérifier les trois champs, puis analyser le message selon la docstring.
```

Lancer `uv run python changelog.py commits.txt 2.0.0 --date 2026-10-09`. Contrôler le changement incompatible `9f2c1ab` et la ligne de fusion rejetée `4a7d6bc`. Conserver un exemple de règle précisée, de plan ajusté ou de proposition corrigée.

Partie 4 — Rendre la tâche réutilisable (10 min)

Créer `.github/prompts/changelog.prompt.md` avec **Chat: New Prompt File**, puis le lancer avec `/changelog` dans la cible **Local**. Ce point de départ peut aider :

```
---
name: changelog
agent: agent
---
Prépare la version ${input:version:2.1.0} depuis commits.txt.
Lance uv run python changelog.py commits.txt ${input:version:2.1.0}.
Montre les lignes ignorées et la section produite avant insertion.
Insère sous le titre de CHANGELOG.md, en conservant les versions précédentes.
Si la version existe déjà, signale-le sans créer de doublon.
```

Relire le fichier de prompt, vérifier la section avant insertion, puis relancer pour la même version et vérifier qu'elle n'est pas dupliquée.

Partie 5 — Rédiger la charte (10 min)

En binôme, remplir `charte_ia_modele.md` et enregistrer `charte_ia.md`. Rédiger trois règles : outil pour une tâche précise, données à exclure d'un prompt, vérification avant de garder du code. Donner un exemple vécu pour chaque règle. Préparer la restitution de deux minutes du bilan collectif.

Partie 6 — Vérifier et expliquer (5 min)

Lancer `uv run pytest -q`. Expliquer une décision prise pendant le projet et la vérification qui la justifie. Montrer le changelog, le fichier de prompt et la charte.

Critères de réussite

- Les 13 tests fournis et vos tests passent, sans modification des signatures ni des tests d'acceptation.
- `spec.md`, `AGENTS.md` et `.github/prompts/changelog.prompt.md` sont relus et utilisables.
- La commande produit le changelog attendu ; `/changelog` ne duplique pas une version.
- `charte_ia.md` contient trois règles avec leurs exemples.

Corrigé

Le dossier `corrige/` contient le script, les tests, la spécification, les règles du projet, le fichier de prompt et un exemple de charte. `corrige/README.md` décrit les vérifications.